B.C.A. Semester V · School of Computer & Systems Sciences

COMPUTER GRAPHICS & MULTIMEDIA

Jaipur National University, Jaipur

Paper Code: BCAMACO501T22 | 3-0-0-3

Complete Topper Notes · Algorithms + Numericals + Diagrams + Revision

Unit I: Intro & Scan Conversion

Unit II: Devices & Clipping

Unit III: Transformations

Unit IV: Multimedia Systems

Unit V: Streams & File Formats

Textbooks: Foley et al. · Steinmetz & Nahrstedt · D.Hearn & Baker · Ranjan Parekh

📑 Table of Contents

1. **Unit I: Introduction & Scan Conversion**
   - Introduction to Computer Graphics
   - Advantages of Interactive Graphics
   - Representative Uses
   - Classification of Applications
   - Hardware & Software Development
   - Conceptual Framework
   - Scan Converting Lines (DDA, Bresenham)
   - Scan Converting Circles (Bresenham's)
   - Scan Converting Ellipses (Midpoint)
2. **Unit II: Display Devices & Clipping**
   - Hardcopy & Display Technologies
   - Raster-Scan & Random-Scan Systems
   - Input Devices & Image Scanners
   - Cohen-Sutherland Line Clipping
   - Cyrus-Beck Algorithm
   - Midpoint Subdivision Algorithm
3. **Unit III: Geometric Transformations**
   - 2D Transformations (Translation, Rotation, Scaling, Reflection, Shearing)
   - Homogeneous Coordinates
   - Composition of Transformations
   - Window-to-Viewport Transformation
   - 3D Transformations
4. **Unit IV: Multimedia**
   - Multimedia Elements
   - System Architecture & Technologies
   - Data Compression
   - Multimedia Database
5. **Unit V: Media Streams & File Formats**
   - Media and Data Streams
   - RTF, TIFF, RIFF, MIDI, JPEG, DIB, MPEG
6. 📐 **Formula Sheet**
7. 🎯 **Master Exam Revision**

I Introduction to Computer Graphics & Scan Conversion

Unit Overview:

- Introduces Computer Graphics, its uses, classification, hardware/software, and the conceptual framework.
- Covers three fundamental scan-conversion algorithms: **Lines** (DDA & Bresenham), **Circles** (Bresenham's), **Ellipses** (Midpoint).
- Exam importance: 🔥 Very High — scan conversion algorithms with numerical examples are guaranteed questions.

1.1 Introduction to Computer Graphics 🔥 Very Important

> **DEFINITION**
>
> **Computer Graphics** is the branch of computer science concerned with the **creation, manipulation, storage, and display** of pictorial data using a computer. It involves converting geometric and visual descriptions into pictures displayed on output devices.

Two Types:

Feature	Interactive Graphics	Non-Interactive (Passive) Graphics
User control	User can interact and modify the display in real time	No user interaction; fixed output
Communication	Two-way (user ↔ system)	One-way (system → user)
Example	CAD systems, video games, GUIs	Plotted graphs, printed charts
Feedback	Immediate visual feedback	No real-time feedback
Complexity	More complex hardware/software	Simpler

1.2 Advantages of Interactive Graphics ☆ Important

- **Better communication:** A picture is worth a thousand words — complex data displayed visually is understood faster.
- **Increased productivity:** Engineers/designers work faster using graphical tools (CAD/CAM).
- **Better visualization:** Scientific and engineering simulations become tangible.
- **Real-time interaction:** User can modify designs on the fly and see immediate results.
- **Education & training:** Simulations (flight simulators, medical training) enable safe practice.
- **Entertainment:** Movies, games, animation — massive industry.
- **Data analysis:** Charts, graphs, maps make data trends visible instantly.
- **Reduced cost:** Digital prototyping replaces expensive physical models.

1.3 Representative Uses of Computer Graphics ☆ Important

Application Area	Example
Design (CAD/CAM)	Mechanical parts, circuit boards, architectural plans
Presentation Graphics	Business charts, slides, infographics
Entertainment	Video games (3D graphics), animated movies (Pixar)
Education & Training	Flight simulators, medical simulators, e-learning
Scientific Visualization	Molecular structure, weather maps, seismic data
Image Processing	Medical imaging (MRI, CT scan), satellite images
Geographic Information	Maps, GIS, GPS visualization
Art & Illustration	Digital painting, photo editing (Photoshop)
User Interfaces	Windows, icons, GUI components

1.4 Classification of Applications ○ Normal

- **Raster graphics applications:** Image editing (pixel-based) — Photoshop, GIMP
- **Vector graphics applications:** Drawing (equation-based) — Illustrator, AutoCAD
- **2D applications:** Flat designs, UI design, charts

- **3D applications:** Solid modeling, animation, rendering
- **Real-time applications:** Games, VR, simulations
- **Non-real-time applications:** Offline rendering, scientific visualization

1.5 Development of Hardware & Software for CG ⭐ Important

Hardware Evolution:

- 1950s: Vector display (CRT) — oscilloscope-type displays
- 1960s: Ivan Sutherland's Sketchpad (1963) — first interactive graphics system
- 1970s: Raster displays, workstations, first GUI
- 1980s: Personal computers, VGA displays, PostScript
- 1990s: GPU development, 3D acceleration, OpenGL
- 2000s+: HD displays, programmable shaders, GPU computing

Key Hardware Components:

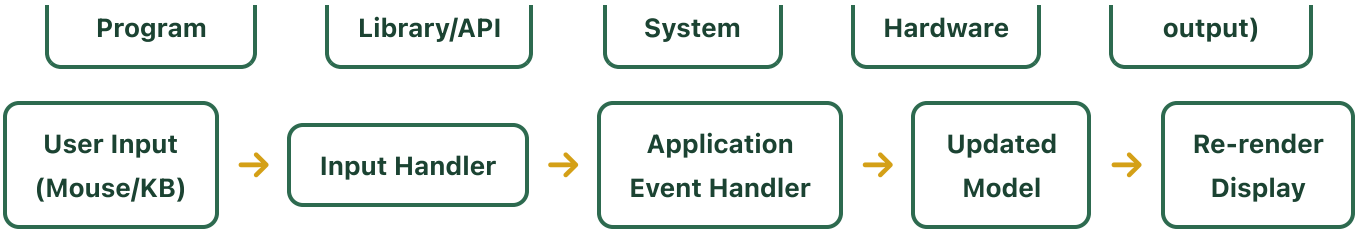
Component	Role
Display Device	Output screen — CRT, LCD, OLED, Plasma
Graphics Card/GPU	Processes and renders graphics (offloads from CPU)
Frame Buffer	Memory holding pixel values for current display
Input Devices	Mouse, keyboard, light pen, joystick, digitizer
Hardcopy Devices	Printer, plotter — for paper output

Software Evolution: Assembly programming → FORTRAN graphics → GKS (Graphical Kernel System, 1985) → PHIGS → OpenGL → DirectX → WebGL → Vulkan

1.6 Conceptual Framework for Interactive Graphics 🔥 Very Important

> **DEFINITION**
> The conceptual framework describes the components and their interactions in an interactive graphics system — from the application model to the physical display.





> **Exam Drawing Tip:** Draw two horizontal pipelines — one for output (App → Library → System → Display) and one for input (User → Input Handler → App → Display). Connect them with "User" as the bridge.

Components:

- **Application Model:** The geometric/visual description of what to display (objects, coordinates)
- **Graphics Library:** API providing drawing primitives (lines, circles, polygons) — e.g., OpenGL
- **Display Processing:** Converts geometric model to pixel data (scan conversion)
- **Frame Buffer:** Stores the final pixel image ready for display
- **Input Handling:** Captures user events and feeds back to the application

Possible Exam Questions:

Explain the conceptual framework for interactive graphics. (5M/10M) | Differentiate interactive and non-interactive graphics. (5M)

1.7 Scan Converting Lines

🔥 Very Important

> **SCAN CONVERSION (RASTERIZATION)**
>
> The process of **determining which pixels on a raster display should be illuminated** to represent a geometric primitive (line, circle, ellipse) as closely as possible. Since a display has discrete pixels, continuous shapes must be approximated.

Algorithm 1: DDA (Digital Differential Analyzer)

DDA LINE ALGORITHM

Purpose: Draw a line from point (x_1, y_1) to (x_2, y_2) on a raster display.

Basic Idea: Calculate intermediate pixel positions by incrementing x by 1 and computing y = y + m (slope), rounding to nearest integer.

Steps:

1. Compute $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$
2. Determine steps = max(|Δx|, |Δy|)
3. Compute increments: x_inc = Δx/steps, y_inc = Δy/steps
4. Start at (x_1, y_1); round and plot each step
5. Repeat steps times, adding increments each time

```
Algorithm DDA_Line(x1, y1, x2, y2):
  dx = x2 - x1
  dy = y2 - y1
  steps = max(|dx|, |dy|)
  x_inc = dx / steps
  y_inc = dy / steps
  x = x1, y = y1
  for i = 1 to steps:
    plot(round(x), round(y))
    x = x + x_inc
    y = y + y_inc
```

🔢 NUMERICAL EXAMPLE — DDA

Given: $P_1 = (0, 0)$, $P_2 = (4, 3)$

Δx = 4 – 0 = 4

Δy = 3 – 0 = 3

Steps = max(4, 3) = 4

x_inc = 4/4 = 1.0

y_inc = 3/4 = 0.75

Step	x (actual)	y (actual)	Plot pixel
0 (start)	0	0	(0, 0)
1	1.0	0.75	(1, 1)
2	2.0	1.50	(2, 2)
3	3.0	2.25	(3, 2)
4	4.0	3.00	(4, 3)

Pixels plotted: (0,0), (1,1), (2,2), (3,2), (4,3)

Advantages: Simple to understand; handles all slopes.

Limitations: Floating-point arithmetic is slow; rounding errors accumulate; not suitable for hardware.

Algorithm 2: Bresenham's Line Algorithm

BRESENHAM'S LINE ALGORITHM

Purpose: Draw a line using only **integer arithmetic** — faster and more accurate than DDA.

Basic Idea: At each step, decide whether to increment y or keep it the same by testing a **decision parameter (d)**.

Derivation (for $0 \le m \le 1$):

1. Given $P_1(x_1,y_1)$ to $P_2(x_2,y_2)$; $dx = x_2 - x_1$, $dy = y_2 - y_1$
2. Initial decision parameter: **$d = 2dy - dx$**
3. For each step (increment x by 1):
   - If **$d < 0$**: choose lower pixel; update $d = d + 2dy$
   - If **$d \ge 0$**: choose upper pixel (increment y); update $d = d + 2(dy - dx)$
4. Plot pixel at current (x, y) and repeat

```
Algorithm Bresenham_Line(x1, y1, x2, y2):
  dx = x2 - x1
  dy = y2 - y1
  d = 2*dy - dx          // initial decision parameter
  x = x1, y = y1
  plot(x, y)
  for x = x1+1 to x2:
    if d < 0:
      d = d + 2*dy       // lower pixel selected
    else:
      y = y + 1
      d = d + 2*(dy - dx) // upper pixel selected
    plot(x, y)
```

🔢 NUMERICAL EXAMPLE — BRESENHAM'S LINE

Given: $P_1 = (0, 0)$, $P_2 = (7, 4)$

dx = 7, dy = 4

d initial = 2(4) − 7 = 1

2dy = 8 | 2(dy−dx) = 2(4−7) = −6

x	y	d	Decision	Plot
0	0	1	Start	(0,0)
1	1	1+(−6)=−5	d≥0 → y+1	(1,1)
2	1	−5+8=3	d<0 → keep y	(2,1)
3	2	3+(−6)=−3	d≥0 → y+1	(3,2)
4	2	−3+8=5	d<0 → keep y	(4,2)
5	3	5+(−6)=−1	d≥0 → y+1	(5,3)
6	3	−1+8=7	d<0 → keep y	(6,3)
7	4	7+(−6)=1	d≥0 → y+1	(7,4)

Pixels: (0,0),(1,1),(2,1),(3,2),(4,2),(5,3),(6,3),(7,4) ✓

Feature	DDA Algorithm	Bresenham's Algorithm
Arithmetic	Floating-point (division)	Integer only (faster)
Speed	Slower	Faster (hardware friendly)
Accuracy	Rounding errors accumulate	More accurate
Complexity	Simple	Slightly more complex
Hardware	Not ideal	Ideal for hardware

1.8 Scan Converting Circles 🔥 Very Important

DEFINITION

Scan-converting a circle means finding the set of pixels that best approximate a circle of given radius r centered at (xc, yc) on a raster display.

Key Insight — 8-Way Symmetry: A circle is symmetric in 8 octants. If one point (x, y) is on the circle, 7 other points are also on it: (y,x), (y,−x), (x,−y), (−x,−y), (−y,−x), (−y,x), (−x,y). This means we only need to compute **⅛ of the circle**.

Bresenham's Circle Algorithm (Midpoint Circle Algorithm)

MIDPOINT CIRCLE ALGORITHM

Purpose: Draw a circle of radius r centered at origin using integer arithmetic.
Basic Idea: Move from (0, r) clockwise in the first octant. At each step, decide whether next pixel is directly right (E) or diagonally down-right (SE) using a decision parameter.
Steps:

1. Set x = 0, y = r
2. Initial decision parameter: **p = 1 − r**
3. Plot the 8 symmetric points using CirclePlotPoints(xc, yc, x, y)
4. While x < y:
   - If p < 0: x = x+1; p = p + 2x + 1 (East pixel)
   - If p ≥ 0: x = x+1; y = y−1; p = p + 2x − 2y + 1 (SE pixel)
5. Plot all 8 symmetric points each iteration

```
Algorithm Midpoint_Circle(xc, yc, r):
  x = 0, y = r
  p = 1 - r              // initial decision parameter
  PlotCirclePoints(xc, yc, x, y)
  while x < y:
    x = x + 1
    if p < 0:
      p = p + 2*x + 1
    else:
      y = y - 1
      p = p + 2*x - 2*y + 1
    PlotCirclePoints(xc, yc, x, y)

PlotCirclePoints(xc, yc, x, y):
  // 8 symmetric points
  plot(xc+x, yc+y);  plot(xc-x, yc+y)
  plot(xc+x, yc-y);  plot(xc-x, yc-y)
  plot(xc+y, yc+x);  plot(xc-y, yc+x)
  plot(xc+y, yc-x);  plot(xc-y, yc-x)
```

🔢 NUMERICAL EXAMPLE — MIDPOINT CIRCLE (R = 6, CENTER = (0,0))

Step	x	y	p	Decision	Points Plotted (1st octant)
Init	0	6	1−6=−5	—	(0,6),(6,0)
1	1	6	−5+2(1)+1=−2	p<0: E	(1,6),(6,1)
2	2	6	−2+2(2)+1=3	p<0: E	(2,6),(6,2)
3	3	5	3+2(3)−2(6)+1=2	p≥0: SE	(3,5),(5,3)
4	4	4	2+2(4)−2(5)+1=1	p≥0: SE	(4,4)
—	x = y = 4, stop (x not < y)				

Using 8-fold symmetry, complete circle points are generated from these few computations.

1.9 Scan Converting Ellipses 🔥 Very Important

DEFINITION

Scan-converting an ellipse means finding pixels that approximate the ellipse with semi-major axis **a** (along x) and semi-minor axis **b** (along y), centered at (xc, yc). Equation: $(x/a)^2 + (y/b)^2 = 1$

4-Way Symmetry: Ellipses have symmetry in 4 quadrants (not 8 like circles). Compute one quadrant, mirror the rest.

Midpoint Ellipse Algorithm

MIDPOINT ELLIPSE ALGORITHM

Key concept: The ellipse is divided into **Region 1** (|slope| < 1, starting at top, moving right) and **Region 2** (|slope| > 1, moving down). Transition occurs when $2b^2x = 2a^2y$.

Ellipse function: $f(x,y) = b^2x^2 + a^2y^2 - a^2b^2$

- $f < 0$ → point inside ellipse
- $f = 0$ → on ellipse
- $f > 0$ → outside ellipse

Region 1 (start at top: x=0, y=b):

- $p_1 = b^2 - a^2b + a^2/4$
- If $p_1 < 0$: x++; $p_1 = p_1 + 2b^2x + b^2$
- If $p_1 \geq 0$: x++; y--; $p_1 = p_1 + 2b^2x - 2a^2y + b^2$
- Continue until $2b^2x \geq 2a^2y$

Region 2 (transition point to (a,0)):

- $p_2 = b^2(x+½)^2 + a^2(y-1)^2 - a^2b^2$
- If $p_2 > 0$: y--; $p_2 = p_2 - 2a^2y + a^2$
- If $p_2 \leq 0$: x++; y--; $p_2 = p_2 + 2b^2x - 2a^2y + a^2$
- Continue until $y = 0$

Exam Tip — Ellipse vs Circle:

- Circle: 8-fold symmetry; one region; initial $p = 1 - r$
- Ellipse: 4-fold symmetry; **two regions**; more complex decision parameters
- Both use midpoint decision — the fundamental principle is the same

🔢 NUMERICAL — MIDPOINT ELLIPSE (A=4, B=3, CENTER ORIGIN)

a=4, b=3 $a^2=16$, $b^2=9$

Region 1 start: x=0, y=3

p_1 initial: $b^2-a^2b+a^2/4 = 9-16(3)+16/4 = 9-48+4 = -35$

x	y	p_1	Decision
0	3	−35	p<0: x++ → $p_1=-35+2(9)(1)+9=-17$
1	3	−17	p<0: x++ → $p_1=-17+2(9)(2)+9=10$
2	3	10	p≥0: x++,y-- → $p_1=10+2(9)(3)-2(16)(3)+9=-5$
3	2	—	Check $2b^2x$ vs $2a^2y$: 2(9)(3)=54 ≥ 2(16)(2)=64? No → still Reg 1

Continue until $2b^2x \geq 2a^2y$, then switch to Region 2. Each (x,y) maps to 4 symmetric points: (±x, ±y).

📌 Quick Revision — Unit I

- CG = creating, displaying visual data using computers
- Interactive CG: user ↔ system; Non-interactive: one-way
- Scan conversion = converting geometric shape to pixels
- DDA: floating-point; steps = max(|Δx|,|Δy|)
- Bresenham Line: integer only; d = 2dy−dx initially
- Bresenham Circle: 8-fold symmetry; p = 1−r
- Ellipse: 4-fold symmetry; 2 regions; midpoint method
- Bresenham faster than DDA (no floating point)
- Sketchpad (1963) = first interactive CG system
- Circle: only compute 1 octant, mirror 7 others

2-MARK QUESTIONS

1. What is computer graphics?
2. What is scan conversion?
3. State advantages of CG.
4. What is 8-fold symmetry?
5. What is DDA?
6. What is the initial dp in Bresenham circle?

5-MARK QUESTIONS

1. Explain DDA algorithm with example.
2. Compare DDA and Bresenham's line algorithm.
3. Explain Bresenham's circle algorithm.
4. Explain the conceptual framework for interactive CG.
5. State uses of computer graphics with examples.

10-MARK QUESTIONS

1. Explain Bresenham's line algorithm with complete derivation and numerical example.
2. Explain midpoint circle algorithm with steps and numerical.
3. Explain midpoint ellipse algorithm for both regions with example.

4. Explain scan conversion of lines comparing DDA and Bresenham's.

🧠 Memory Tricks — Unit I

- DDA = **D**ivide-**D**ivide-**A**dd (lots of division, floating point)
- Bresenham = **B**est Integer-only algorithm; **B**ig advantage over DDA
- Circle: p initial = 1–r; if p<0 → **E**ast; if p≥0 → **S**outh**E**ast
- Ellipse: 2 regions — when $2b^2x \geq 2a^2y$ → switch from Region 1 to Region 2
- Symmetry: Circle=8 octants; Ellipse=4 quadrants

II Display Technologies, Input Devices & Clipping

Unit Overview:

- Covers display hardware (Raster, Random-Scan), input devices, and three important **line clipping algorithms**.
- Exam focus: Cohen-Sutherland algorithm (with region codes and numerical) is a guaranteed 10-mark question.

2.1 Hardcopy Technologies ⭐ Important

DEFINITION

Hardcopy devices produce permanent output on physical media (paper, film) that exists independently of the computer.

Device	How it Works	Example / Use
Dot Matrix Printer	Pins strike ribbon to print dots	Receipts, old office use
Inkjet Printer	Sprays tiny ink droplets on paper	Home/office color printing
Laser Printer	Laser+toner on drum → paper	Office high-speed printing
Plotter	Pen draws vectors on paper	CAD drawings, maps, blueprints
Electrostatic Plotter	Static charges attract toner	Large format engineering drawings
Thermal Printer	Heat-sensitive paper or wax transfer	ATM receipts, medical charts

2.2 Display Technologies 🔥 Very Important

Technology	Principle	Advantages	Limitations
CRT (Cathode Ray Tube)	Electron beam hits phosphor-coated screen	High contrast, fast response, cheap	Heavy, bulky, high power, radiation
LCD (Liquid Crystal)	Liquid crystals block/allow backlight	Flat, light, energy-efficient	Limited viewing angle, slower response
Plasma	Ionized gas glows per pixel	Large screens, good contrast	Heavy, expensive, burn-in
LED/OLED	LEDs/organic LEDs emit light directly	Thin, vibrant colors, true black	OLED can burn-in; expensive
DVST (Direct View Storage Tube)	Maintains image by charged mesh	No refresh needed	Cannot erase selectively

2.3 Raster-Scan Display System 🔥 Very Important

> DEFINITION
>
> A **Raster-Scan Display** draws the screen by scanning the electron beam horizontally left-to-right, top-to-bottom, line by line (like a TV). Each scanline is made up of discrete **pixels** stored in a **frame buffer**.



Key Properties:

- **Resolution:** e.g., 1920×1080 pixels (Full HD)
- **Refresh rate:** Typically 60–120 Hz (frames per second); prevents flicker
- **Frame buffer:** Stores intensity (color) value for every pixel
- **Pixel (picture element):** Smallest addressable unit of the display
- **Bit planes:** Number of bits per pixel (1 bit = B&W; 8 bits = 256 colors; 24 bits = True Color)

2.4 Video Controller ☆ Important

> **DEFINITION**
>
> The **Video Controller** (or display controller) reads pixel values from the frame buffer and generates the analog/digital signals required to drive the display device at the correct timing.

Functions:

- Reads frame buffer in scan order (row by row)
- Converts pixel data to analog video signal (for CRT)
- Generates sync signals: horizontal sync (end of row) and vertical sync (end of frame)
- Manages refresh timing to prevent flicker
- Contains a **color lookup table (CLUT/palette)** for indexed color modes

2.5 Random-Scan Display Processor ☆ Important

> **DEFINITION**
>
> A **Random-Scan (Vector) Display** draws only the lines and curves that form the picture — the electron beam is directed anywhere on the screen in any order (randomly), tracing geometric primitives directly.

Feature	Raster-Scan Display	Random-Scan (Vector) Display
Drawing method	Scans all pixels line by line	Draws only defined geometric shapes
Output quality	Jagged edges (aliasing) possible	Smooth, sharp lines
Memory	Frame buffer (all pixels stored)	Display list (only commands stored)
Filled areas	Easy — fill pixels	Difficult — must draw many lines
Refresh	Fixed rate (60 Hz), independent of picture	Rate depends on number of lines; may flicker for complex pictures
Resolution	Limited by pixel grid	Infinite (vector-based)
Application	General-purpose (games, UI)	CAD, oscilloscopes, early graphics
Cost	Less expensive	More expensive

2.6 Input Devices & Image Scanners ⭐ Important

Device	Type	How It Works
Keyboard	String/valuator	Key matrix; generates character codes
Mouse	Locator	Tracks 2D movement; optical or mechanical
Trackball	Locator	Ball rolled to control cursor
Joystick	Locator/valuator	Stick tilted to indicate direction
Light Pen	Locator/pick	Senses light from CRT beam; identifies pixel
Touch Screen	Locator	Resistive, capacitive or infrared sensing
Digitizer/Tablet	Locator	Stylus position detected electromagnetically
Image Scanner	—	Converts physical images to digital bitmap using CCD sensor

Image Scanner:

- Uses a **CCD (Charge Coupled Device)** array to convert reflected light to digital values
- Output: digital bitmap at specified **dpi (dots per inch)** resolution
- Types: Flatbed, handheld, drum, document (ADF)

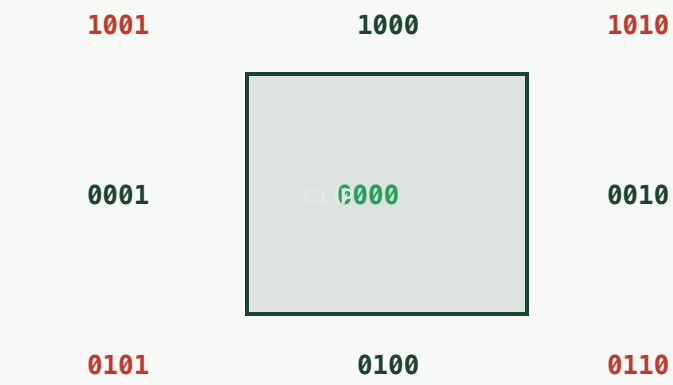
2.7 Cohen-Sutherland Line Clipping Algorithm 🔥 Very Important

> **CLIPPING**
>
> **Clipping** is the process of removing portions of graphical primitives (lines, polygons) that lie outside a specified **clipping window (viewport)**. Only the visible portion is drawn.

> **COHEN-SUTHERLAND ALGORITHM**
>
> An efficient line clipping algorithm that uses a **4-bit region code (outcode)** assigned to each endpoint to quickly determine whether a line segment is inside, outside, or spanning the clipping window.

Region Codes (Outcodes)

The clipping window divides the 2D plane into 9 regions. Each region gets a 4-bit code: **[Top | Bottom | Right | Left]**



Bit 4=Top, Bit 3=Bottom, Bit 2=Right, Bit 1=Left
0000 = Inside (visible)

> **Exam Drawing Tip:** Draw a rectangle (clip window). Divide surrounding space into 9 regions. Label centre 0000 (green). Top row: 1001,1000,1010. Middle row: 0001,[0000],0010. Bottom row: 0101,0100,0110. Add bit legend below.

Bit Assignment Rule

Bit	Position (Bit no.)	Set to 1 if...
Bit 1 (LEFT)	1 (LSB)	x < xmin
Bit 2 (RIGHT)	2	x > xmax
Bit 3 (BOTTOM)	3	y < ymin
Bit 4 (TOP)	4 (MSB)	y > ymax

COHEN-SUTHERLAND ALGORITHM STEPS

1. Compute outcode for both endpoints P_1 and P_2
2. **Trivial Accept:** If $code_1$ OR $code_2$ = 0000 (both inside) → draw line, done
3. **Trivial Reject:** If $code_1$ AND $code_2 \neq 0000$ (both outside same region) → discard line, done
4. **Otherwise (clipping needed):**
   - Select the endpoint with non-zero outcode (outside point)
   - Find the boundary bit that is set (Top/Bottom/Left/Right)
   - Compute intersection with that boundary using line equation
   - Replace the outside endpoint with the intersection point
   - Recompute outcode and repeat from step 2

Intersection Calculations

INTERSECTION FORMULAS

Line: from $P_1(x_1,y_1)$ to $P_2(x_2,y_2)$, slope $m = (y_2-y_1)/(x_2-x_1)$

Left boundary (x = xmin): $y = y_1 + m(xmin - x_1)$

Right boundary (x = xmax): $y = y_1 + m(xmax - x_1)$

Bottom boundary (y = ymin): $x = x_1 + (ymin - y_1)/m$

Top boundary (y = ymax): $x = x_1 + (ymax - y_1)/m$

🔢 NUMERICAL — COHEN-SUTHERLAND

Window: xmin=10, xmax=50, ymin=10, ymax=40

Line: P_1=(0,15), P_2=(60,45)

Slope m: (45−15)/(60−0) = 30/60 = 0.5

Step 1 — Compute outcodes:

- P_1=(0,15): x<xmin → LEFT bit set → code = **0001**
- P_2=(60,45): x>xmax → RIGHT; y>ymax → TOP → code = **1010**

Step 2 — Tests:

- Trivial Accept? 0001 OR 1010 = 1011 ≠ 0000 → *No*
- Trivial Reject? 0001 AND 1010 = 0000 → *No*
- → Must clip

Step 3 — Clip P_1 (code=0001) against LEFT boundary (x=10):

- y = 15 + 0.5(10−0) = 15 + 5 = **20**
- New P_1 = (10, 20), code = 0000 (inside)

Step 4 — Clip P_2 (code=1010) against TOP boundary (y=40):

- x = 0 + (40−15)/0.5 = 25/0.5 = **50**
- New P_2 = (50, 40), code = 0000 (inside)

Clipped line: from (10, 20) to (50, 40) ✓

📝 10-MARK ANSWER STRATEGY: COHEN-SUTHERLAND

1. **Define clipping** and why it's needed.
2. **Explain region codes** — 4-bit scheme, 9 regions, diagram.
3. **State three cases:** trivial accept, trivial reject, clip required.
4. **Explain intersection formulas.**
5. **Write pseudocode** for the algorithm.
6. **Solve numerical example** step by step.
7. **State advantages**: fast for trivially in/out lines.
8. **State limitations**: inefficient for many clipping iterations.

Marks tip: Region code diagram = 2 marks. Algorithm steps = 3 marks. Numerical = 3 marks. Advantages + Conclusion = 2 marks.

2.8 Cyrus-Beck Algorithm 🔥 Very Important

DEFINITION

The **Cyrus-Beck algorithm** is a parametric line-clipping algorithm that works with any **convex polygon** clipping window (not just rectangles). It uses the **parametric form of a line** to find intersection parameters.

Parametric Line Equation:

KEY FORMULA

$P(t) = P_1 + t(P_2 - P_1)$ for $t \in [0, 1]$

Where $t=0$ gives P_1, $t=1$ gives P_2

Intersection with edge i:

$$t = N_i \cdot (PE_i - P_1) / [N_i \cdot (P_2 - P_1)]$$

- $\mathbf{N_i}$ = outward normal of clipping edge i
- $\mathbf{PE_i}$ = any point on clipping edge i
- $\mathbf{N_i \cdot (P_2 - P_1)}$ = denominator D
- $\mathbf{N_i \cdot (PE_i - P_1)}$ = numerator W

CYRUS-BECK STEPS

1. Represent line as $P(t) = P_1 + t(P_2 - P_1)$ for $t \in [0,1]$
2. Initialize: tL = 0, tU = 1 (entering and leaving parameters)
3. For each clipping edge i:
   - Compute $D = N_i \cdot (P_2 - P_1)$
   - Compute $W = N_i \cdot (PE_i - P_1)$
   - $t = W / D$
   - If $D < 0$: line enters from this side → $tL = \max(tL, t)$
   - If $D > 0$: line exits from this side → $tU = \min(tU, t)$
   - If $D = 0$ and $W < 0$: line is parallel and outside → reject
4. If $tL > tU$ → reject (no visible segment)
5. Else → clip P_1 to $P(tL)$ and P_2 to $P(tU)$

Feature	Cohen-Sutherland	Cyrus-Beck
Window shape	Rectangular only	Any convex polygon
Method	Region codes (bit-based)	Parametric (dot product)
Efficiency	Fast for simple cases (trivial tests)	Uniform computation for all cases
Math complexity	Simpler	More mathematical (dot products)
Best for	Mostly inside lines	General convex windows

2.9 Midpoint Subdivision Algorithm ☆ Important

DEFINITION

A line clipping algorithm that **recursively bisects the line segment** at its midpoint until the segment is either trivially accepted (both inside) or trivially rejected (both outside same region), or the segment becomes very small (below a threshold).

MIDPOINT SUBDIVISION STEPS

1. Given segment P_1P_2, compute outcodes for P_1 and P_2
2. If trivially accept (both inside) → draw segment
3. If trivially reject (AND of codes ≠ 0) → discard
4. Otherwise:
   - Compute midpoint $M = ((x_1+x_2)/2, (y_1+y_2)/2)$
   - Recursively apply algorithm to P_1M and MP_2
5. Stop recursion when segment length < threshold (ε)

Feature	Cohen-Sutherland	Midpoint Subdivision
Method	Analytical intersection	Binary subdivision
Precision	Exact intersection	Approximate (limited by ε)
Math required	Line equations needed	Only midpoint calculation
Performance	Fewer iterations	More iterations (recursive)
Suitable for	Standard clipping	Hardware with simple ops

📌 Quick Revision — Unit II

- Raster-Scan: row-by-row pixel scanning; uses frame buffer
- Random-Scan: draws geometric primitives directly; no pixel grid
- Frame buffer = memory storing all pixel values
- Video controller: reads frame buffer → drives display
- Clipping = removing parts outside the clip window
- Cohen-Sutherland: 4-bit outcode; trivial accept/reject; then clip
- Region codes: LEFT=0001, RIGHT=0010, BOTTOM=0100, TOP=1000
- Inside = 0000; trivial accept = both 0000; reject = AND ≠ 0
- Cyrus-Beck: parametric; $P(t)=P_1+t(P_2-P_1)$; any convex window
- Midpoint: recursive bisection at midpoint; uses trivial tests

2-MARK QUESTIONS

1. What is clipping?
2. What is region code in Cohen-Sutherland?
3. What is trivial acceptance?
4. What is a raster-scan display?
5. What is a frame buffer?
6. What is midpoint subdivision?

5-MARK QUESTIONS

1. Compare raster-scan and random-scan displays.
2. Explain Cohen-Sutherland region codes.
3. Compare Cohen-Sutherland and Cyrus-Beck.
4. Explain midpoint subdivision algorithm.
5. Explain input devices for computer graphics.

10-MARK QUESTIONS

1. Explain Cohen-Sutherland line clipping with region codes, algorithm and numerical example.
2. Explain Cyrus-Beck parametric clipping algorithm.
3. Explain and compare raster and random-scan display systems with diagrams.

🧠 Memory Tricks — Unit II

- Region codes — **LRBT** (Left, Right, Bottom, Top) or remember **"Left Right Babies Topping"**
- Bit positions: L=1, R=2, B=4(0100), T=8(1000) — powers of 2
- Trivial Accept: both codes = 0; Trivial Reject: AND ≠ 0
- Cyrus-Beck = **P**arametric + **D**ot product → "**PD**" method
- Raster = like TV (line by line); Random = like drawing board (draw anywhere)

III Geometric Transformations

Unit Overview:

- Covers 2D/3D geometric transformations — Translation, Rotation, Scaling, Reflection, Shearing.
- Explains Homogeneous Coordinates, matrix composition, Window-to-Viewport transformation.
- Exam importance: 🔥 Very High — transformation matrices, composition, and W-to-V numericals are extremely common.

3.1 2D Transformations 🔥 Very Important

DEFINITION

A **geometric transformation** changes the position, orientation, or size of an object. In 2D, we transform the coordinates (x, y) of points to new coordinates (x', y').

1. Translation

DEFINITION

Moving an object from one position to another by adding displacement values.

TRANSLATION FORMULA

$x' = x + tx \quad y' = y + ty$

Where tx, ty are translation distances along x and y axes.

Matrix form: `P' = P + T`

EXAMPLE — TRANSLATION

Point P:	(3, 2)
Translation:	tx=4, ty=3
Result P':	x'=3+4=7, y'=2+3=5 → P'=(7,5)

2. Rotation

DEFINITION

Rotating an object by angle θ about the origin (counterclockwise positive).

ROTATION FORMULA (ABOUT ORIGIN)

$x' = x \cdot \cos\theta - y \cdot \sin\theta$

$y' = x \cdot \sin\theta + y \cdot \cos\theta$

Rotation matrix:

$$R(\theta) = \begin{vmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{vmatrix}$$

EXAMPLE — ROTATION

Point P:	(1, 0)
Angle θ:	90°
$\cos 90° = 0$, $\sin 90° = 1$	
x':	$1 \times 0 - 0 \times 1 = 0$
y':	$1 \times 1 + 0 \times 0 = 1$

P' = (0, 1) — point rotated 90° CCW ✓

3. Scaling

DEFINITION

Changing the size of an object by multiplying coordinates by scale factors.

SCALING FORMULA

$x' = x \cdot Sx \quad y' = y \cdot Sy$

Sx = scale factor along x; Sy = scale factor along y

If Sx = Sy → **uniform scaling**; otherwise **differential scaling**

```
S = | Sx  0  |
    |  0  Sy |
```

EXAMPLE — SCALING

Point P: (2, 3)

Scale: Sx=2, Sy=3

Result: x'=2×2=4, y'=3×3=9 → P'=(4,9)

4. Reflection

DEFINITION

Producing a mirror image of an object about a specified axis.

Reflection about	x' =	y' =	Matrix
X-axis	x	−y	|1,0; 0,−1|
Y-axis	−x	y	|−1,0; 0,1|
Origin	−x	−y	|−1,0; 0,−1|
Line y=x	y	x	|0,1; 1,0|

5. Shearing

DEFINITION

Distorting an object by sliding layers in one direction proportional to distance from a reference axis.

SHEARING FORMULAS

X-direction shear: x' = x + Shx·y y' = y

Y-direction shear: x' = x y' = y + Shy·x

```
X-shear: |1   Shx|    Y-shear: | 1    0|
         |0    1 |             |Shy   1|
```

3.2 Homogeneous Coordinates & Matrix Representation 🔥 Very Important

WHY HOMOGENEOUS COORDINATES?

Standard 2D matrices for Scaling and Rotation can be combined by multiplication. But **Translation** is an addition — it cannot be represented as a 2×2 matrix multiplication. To unify all transformations into a single matrix multiplication, we use **Homogeneous Coordinates**.

Homogeneous Coordinate: A 2D point (x, y) is represented as a 3D point **(x, y, 1)** — adding a third coordinate w=1.

HOMOGENEOUS MATRICES (3×3) FOR 2D TRANSFORMATIONS

Transformation	3×3 Homogeneous Matrix

Translation
by (tx, ty)
	`|1  0  tx|`
	`|0  1  ty|`
	`|0  0   1|`

Rotation
by θ (CCW)
	`|cosθ  -sinθ  0|`
	`|sinθ   cosθ  0|`
	`|  0      0   1|`

Scaling
by Sx, Sy
	`|Sx  0   0|`
	`| 0  Sy  0|`
	`| 0   0  1|`

Reflection
(about X-axis)
	`|1   0  0|`
	`|0  -1  0|`
	`|0   0  1|`

Shear X	`|1  Shx  0|`
	`|0   1   0|`
	`|0   0   1|`

Application: [x', y', 1] = [x, y, 1] × M (row-vector convention)

Key Advantage: With homogeneous coordinates, a sequence of transformations can be combined into a single 3×3 matrix by multiplying the individual matrices — one matrix multiplication for any sequence of T, R, S.

3.3 Composition of 2D Transformations 🔥 Very Important

DEFINITION

Composition (concatenation) is the process of combining multiple transformations into a single transformation matrix by multiplying their matrices in order.

Order Matters: Matrix multiplication is not commutative. $T_1 \times T_2 \neq T_2 \times T_1$ in general.

EXAMPLE — ROTATION ABOUT AN ARBITRARY POINT P(PX,PY)

To rotate by θ about point P (not origin):

1. Translate P to origin: T(−px, −py)
2. Rotate by θ: R(θ)
3. Translate back: T(px, py)

Combined matrix: M = T(−px,−py) × R(θ) × T(px,py)

🔢 NUMERICAL — COMPOSITION (TRANSLATE THEN SCALE)

Point P: (2, 3)

Step 1: Translate by tx=1, ty=2

Step 2: Scale by Sx=2, Sy=2

Composition matrix M = T × S:

```
T = |1 0 1|     S = |2 0 0|
    |0 1 2|         |0 2 0|
    |0 0 1|         |0 0 1|

M = T × S = |2 0 1|
            |0 2 2|
            |0 0 1|

P_h = [2 3 1]
P'_h = [2 3 1] × M = [2×2+0+0, 0+3×2+0, 0+0+1] = [4+0+1×(tx×Sx), ...]

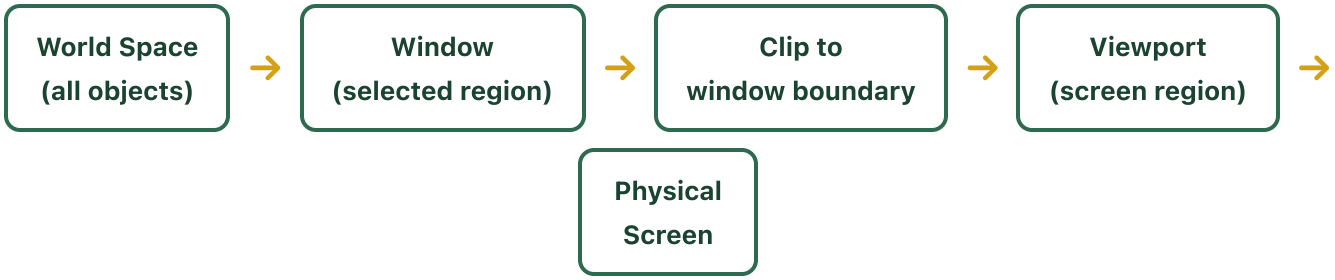
Directly: Translate first: P₁=(2+1,3+2)=(3,5)
Then Scale: P'=(3×2,5×2)=(6,10)
```

Final point P' = (6, 10)

3.4 Window-to-Viewport Transformation 🔥 Very Important

DEFINITION

The **Window** is a rectangular region in **world coordinates** that the user wants to display. The **Viewport** is the rectangular region on the **screen (device coordinates)** where the window content is mapped. The **Window-to-Viewport transformation** maps each point in the window to a corresponding point in the viewport.



WINDOW-TO-VIEWPORT TRANSFORMATION FORMULAS

Window: (wxmin, wymin) to (wxmax, wymax)
Viewport: (vxmin, vymin) to (vxmax, vymax)

sx = (vxmax – vxmin) / (wxmax – wxmin) [x scale]
sy = (vymax – vymin) / (wymax – wymin) [y scale]

x_viewport = vxmin + (x_world – wxmin) × sx
y_viewport = vymin + (y_world – wymin) × sy

🔢 **NUMERICAL — WINDOW TO VIEWPORT**

Window:	wxmin=0, wxmax=100, wymin=0, wymax=80
Viewport:	vxmin=10, vxmax=210, vymin=10, vymax=170
World point:	P=(50, 40)
sx =	(210−10)/(100−0) = 200/100 = 2
sy =	(170−10)/(80−0) = 160/80 = 2
xv =	10 + (50−0)×2 = 10+100 = 110
yv =	10 + (40−0)×2 = 10+80 = 90

Viewport point = (110, 90) ✓

3.5 Matrix Representation of 3D Transformations ⭐ Important

DEFINITION

3D transformations operate on points (x, y, z) in 3D space. In homogeneous coordinates, a point is (x, y, z, 1) and matrices are 4×4.

3D HOMOGENEOUS MATRICES (4×4)

Transformation	Matrix

Translation
T(tx,ty,tz)
	`|1 0 0 tx|`
	`|0 1 0 ty|`
	`|0 0 1 tz|`
	`|0 0 0 1|`

Scaling
S(Sx,Sy,Sz)
	`|Sx 0 0 0|`
	`| 0 Sy 0 0|`
	`| 0 0 Sz 0|`
	`| 0 0 0 1|`

Rotation about Z-axis by θ	`|cosθ -sinθ 0 0|`
	`|sinθ cosθ 0 0|`
	`| 0 0 1 0|`
	`| 0 0 0 1|`

Rotation about X-axis by θ	`|1 0 0 0|`
	`|0 cosθ -sinθ 0|`
	`|0 sinθ cosθ 0|`
	`|0 0 0 1|`

Rotation about Y-axis by θ	`| cosθ 0 sinθ 0|`
	`| 0 1 0 0|`
	`|-sinθ 0 cosθ 0|`
	`| 0 0 0 1|`

Feature	2D Transformations	3D Transformations
Coordinate	(x, y)	(x, y, z)
Homogeneous	(x, y, 1) — 3D vector	(x, y, z, 1) — 4D vector
Matrix size	3×3	4×4
Rotation axes	About Z-axis only	About X, Y, or Z axis
Complexity	Simpler	More complex; Gimbal lock possible

Transformation as Change in Coordinate System: Instead of moving the object, we can equivalently move the coordinate system in the opposite direction. This "change of frame" perspective is useful in 3D viewing pipelines.

📌 Quick Revision — Unit III

- Translation: x'=x+tx, y'=y+ty
- Rotation: x'=x·cosθ−y·sinθ; y'=x·sinθ+y·cosθ
- Scaling: x'=x·Sx; y'=y·Sy
- Reflection about X: (x,−y); about Y: (−x,y)
- Shear-X: x'=x+Shx·y; y'=y
- Homogeneous: add w=1; makes Translation a matrix multiply
- 2D matrices: 3×3; 3D matrices: 4×4
- Composition: multiply matrices left-to-right in order
- Rotation about pivot P: Translate-to-O → Rotate → Translate-back
- W-to-V: sx=(vxmax−vxmin)/(wxmax−wxmin); xv=vxmin+(xw−wxmin)·sx

2-MARK QUESTIONS

1. What is geometric transformation?
2. Why are homogeneous coordinates used?
3. What is a viewport?
4. Define translation with formula.
5. What is scaling?
6. What is composition of transformations?

5-MARK QUESTIONS

1. Explain all 2D transformations with matrices.
2. Explain homogeneous coordinates.
3. Explain Window-to-Viewport transformation with formula.
4. Explain composition of transformations with example.

10-MARK QUESTIONS

1. Explain 2D transformations with homogeneous matrices and numerical examples.
2. Explain Window-to-Viewport transformation pipeline with derivation and example.
3. Explain 3D transformation matrices for translation, rotation, scaling.

5. Compare 2D and 3D transformations.

🧠 Memory Tricks — Unit III

- 2D transformations: **T-R-S-R-S** = Translation, Rotation, Scaling, Reflection, Shearing
- Rotation matrix: **"cos top-left, sin bottom-left, –sin top-right, cos bottom-right"**
- Homogeneous: add a **1** at the end — (x,y,1) for 2D, (x,y,z,1) for 3D
- Translation in 3×3: last COLUMN has tx, ty (column vector convention) or last ROW has tx, ty (row vector convention)
- W-to-V: **S**cale then **S**hift = sx = viewport_width/window_width

IV Multimedia Systems

Unit Overview:

- Introduces multimedia — its elements, applications, system architecture, evolving technologies, standards, and data compression.
- Exam importance: 🔥 Very High — Multimedia elements, architecture, and data compression (lossless vs lossy) are standard exam questions.

4.1 Multimedia Elements 🔥 Very Important

DEFINITION

Multimedia is the integration of **multiple types of media** — text, graphics, audio, video, and animation — in a single application or system, typically controlled by a computer. The key is **interaction** and **integration**.

🧠 **Mnemonic for Multimedia Elements: T-I-A-V-A**
Text, **I**mage/Graphics, **A**udio, **V**ideo, **A**nimation

Element	Definition	Formats	Size	Examples
Text	Alphanumeric characters; most basic form of information	TXT, RTF, PDF, HTML	Smallest	Articles, e-books, menus
Image/Graphics	Still visual; can be raster (pixel) or vector	JPEG, PNG, BMP, GIF, TIFF	Medium	Photos, logos, illustrations
Audio	Digital representation of sound — sampled and quantized	MP3, WAV, MIDI, AAC	Medium	Music, speech, sound effects
Video	Sequence of images (frames) played at speed (fps)	MP4, AVI, MOV, MPEG	Largest	Movies, tutorials, news
Animation	Sequence of frames giving illusion of movement; can be computer-generated	GIF, SWF, WebM	Variable	Cartoons, UI effects, simulations

Key Parameters of Audio:

- **Sampling rate:** Number of samples per second (Hz) — CD quality = 44,100 Hz
- **Bit depth:** Bits per sample — 8-bit, 16-bit, 24-bit
- **Channels:** Mono (1), Stereo (2), Surround (5.1, 7.1)
- Audio file size = Sampling rate × Bit depth × Channels × Duration / 8 (bytes)

Key Parameters of Video:

- **Frame rate:** fps (frames per second) — TV: 25/30 fps, Film: 24 fps
- **Resolution:** Width × Height pixels — 1920×1080 (Full HD)
- **Bit rate:** Data transmitted per second (Mbps)
- Uncompressed video size is huge — hence compression is essential

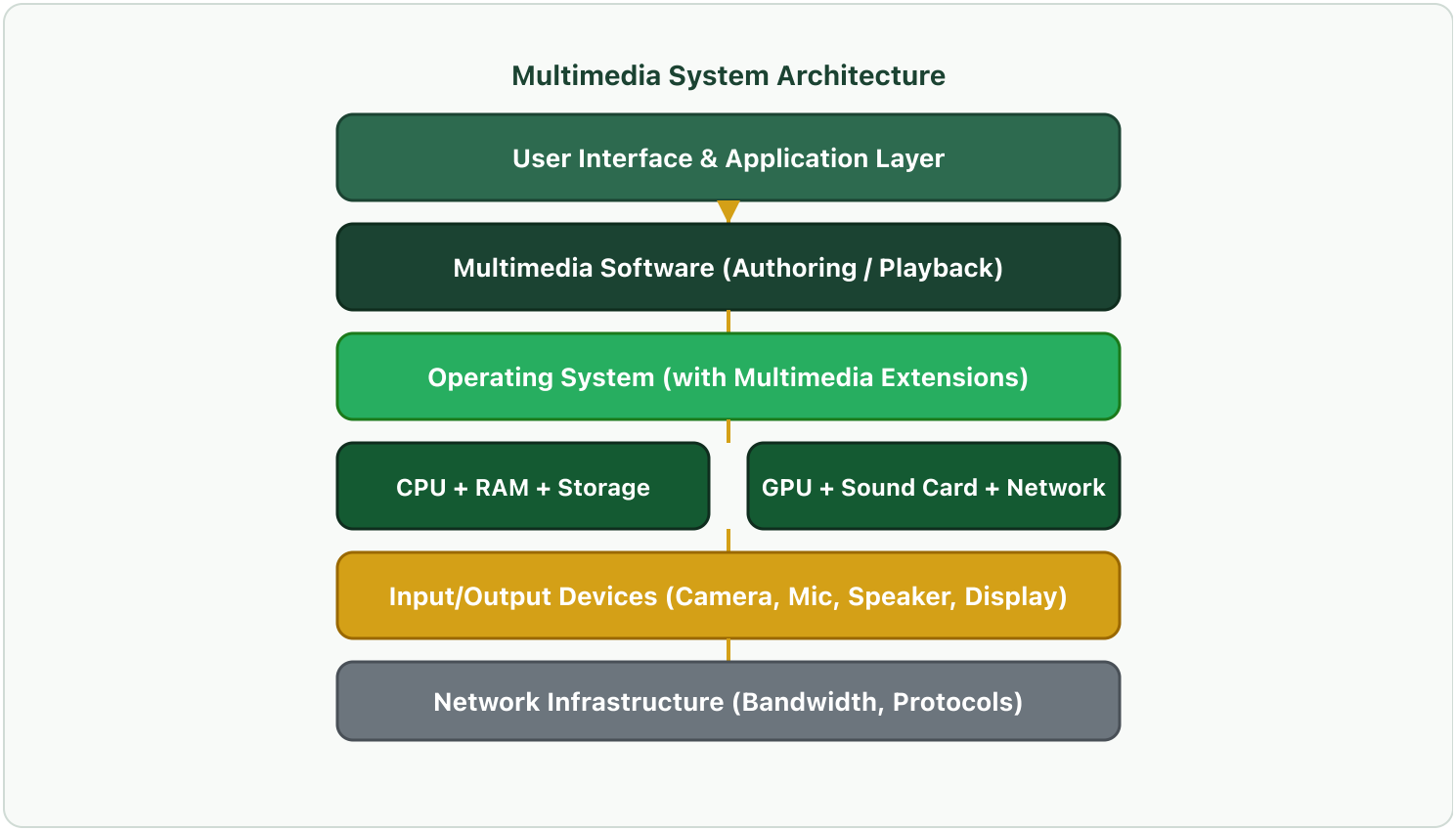
4.2 Multimedia Applications ⭐ Important

Application Area	Example
Education / e-Learning	Online courses (Coursera), interactive textbooks, simulations
Entertainment	Games, streaming (Netflix, YouTube), cinema
Business / Presentations	PowerPoint, product demos, kiosks
Healthcare	Medical imaging, patient education, telemedicine
Advertising	Digital banners, video ads, interactive displays
Communication	Video conferencing (Zoom), VoIP, social media
Publishing	Digital magazines, e-books, interactive PDFs
Training & Simulation	Flight simulators, military training, VR environments

4.3 Multimedia System Architecture 🔥 Very Important

DEFINITION

A **multimedia system** is a computer system capable of processing, storing, generating, and delivering multimedia content in an integrated and synchronized manner.



> **Exam Drawing Tip:** Draw 6 horizontal stacked rectangles: User Interface → MM Software → OS → Hardware (2 boxes side by side) → I/O Devices → Network. Add downward arrows between layers.

Key Components Explained:

- **CPU:** Processes media data; speed critical for real-time video/audio
- **GPU:** Hardware-accelerated graphics rendering and video decoding
- **RAM:** Holds active multimedia data; must be sufficient for buffers
- **Storage:** HDD/SSD — large capacity needed for media files
- **Sound Card:** Digitizes audio (ADC) and plays back (DAC)
- **Network:** For streaming — bandwidth must match bit rate of content
- **Operating System:** Must support real-time scheduling, device drivers, media APIs

4.4 Evolving Technologies for Multimedia Systems ☆ Important

- **VR (Virtual Reality):** Immersive 3D environment — Oculus, HTC Vive
- **AR (Augmented Reality):** Overlays digital information on real world — Pokémon GO
- **MR (Mixed Reality):** Blend of real and virtual — Microsoft HoloLens
- **4K/8K Video:** Ultra-high resolution displays; huge bandwidth/storage needs
- **Streaming Technologies:** HLS, DASH, WebRTC for live/on-demand streaming

- **Cloud Multimedia:** Cloud storage, rendering farms, CDN delivery
- **AI in Multimedia:** Deepfakes, super-resolution, automatic captioning
- **360° Video:** Spherical video for VR headsets

4.5 Multimedia Data Interface Standards ⭐ Important

Standard	Purpose	Example
TWAIN	Interface between imaging hardware and software	Scanner drivers
DirectX	Windows multimedia API (graphics, audio, input)	PC games
OpenGL	Cross-platform 3D graphics API	3D applications, games
MPEG	Video/audio compression standards	MP3, MP4, DVD, Blu-ray
MIDI	Musical Instrument Digital Interface protocol	Digital music keyboards
MCI	Media Control Interface — Windows media device control	Playing audio/video in Windows
JPEG	Image compression standard	Digital photos

4.6 Need for Data Compression 🔥 Very Important

> **WHY COMPRESSION?**
>
> Multimedia data is enormous. A 1-minute uncompressed Full HD video (1920×1080, 30fps, 24-bit color) ≈ **3.7 GB**. Without compression, storage and transmission would be impractical. Compression reduces the file size by removing redundant or perceptually unimportant data.

Types of Redundancy:

- **Spatial redundancy:** Adjacent pixels in an image often have similar colors (can be encoded more compactly)
- **Temporal redundancy:** Adjacent frames in video are often similar (only the differences change)
- **Psychovisual redundancy:** Human eye cannot perceive all fine detail — some information can be safely discarded
- **Coding redundancy:** Not all values occur equally — frequent values can use shorter codes

Feature	Lossless Compression	Lossy Compression
Definition	Reconstructs exact original data	Approximates original; some data discarded
Quality	Perfect — no quality loss	Quality reduced (but often imperceptible)
Compression ratio	Lower (2:1 to 4:1)	Higher (10:1 to 100:1)
Use for	Text, medical images, archival	Photos, audio, video for general use
Algorithms	Huffman coding, RLE, LZW, PNG	JPEG, MP3, MPEG, AAC
Reversible?	Yes — original can be fully restored	No — original cannot be fully restored

Common Compression Techniques:

- **RLE (Run-Length Encoding):** Replace repeated values with count. E.g., AAAABBB → 4A3B. Good for simple images.
- **Huffman Coding:** Frequent symbols get shorter bit codes; rare symbols get longer codes.
- **LZW (Lempel-Ziv-Welch):** Dictionary-based; replaces repeated strings with shorter codes. Used in GIF, TIFF.
- **DCT (Discrete Cosine Transform):** Used in JPEG and MPEG; converts spatial data to frequency domain; high-frequency components (less visible) discarded.
- **Wavelet Transform:** Used in JPEG 2000; better quality at high compression ratios.

4.7 Multimedia Database ☆ Important

> DEFINITION
>
> A **Multimedia Database (MMDB)** is a database system capable of storing, managing, and retrieving various types of multimedia data — text, images, audio, video, and animation — along with their metadata.

Challenges of MMDB:

- **Large file sizes:** Requires terabytes of storage
- **Content-based retrieval:** Query by image/audio features, not just text keywords
- **Streaming:** Continuous delivery with real-time constraints
- **Metadata management:** Indexing properties (resolution, duration, tags)
- **Synchronization:** Audio-video sync must be maintained

Features: Content-based indexing, similarity search, temporal data model, support for large binary objects (BLOBs/CLOBs)

📌 Quick Revision — Unit IV

- Multimedia = Text + Image + Audio + Video + Animation (T-I-A-V-A)
- Multimedia = integration + synchronization + interactivity
- Audio: sampling rate × bit depth × channels = quality
- Video: frame rate × resolution = data size
- MM Architecture: UI → Software → OS → Hardware → I/O → Network
- Lossless: exact recovery; Lossy: approximate; smaller files
- Compression needed: 1 min HD video ≈ 3.7 GB uncompressed
- RLE: AAABBB→3A3B; Huffman: frequent=short codes; LZW: dictionary
- JPEG, MPEG, MP3 = lossy; PNG, TIFF (LZW), GIF = lossless
- MMDB: stores text+image+audio+video; supports content-based retrieval

2-MARK QUESTIONS

1. Define multimedia.
2. List multimedia elements.
3. What is lossless compression?
4. What is a multimedia database?
5. What is temporal redundancy?
6. Define sampling rate.

5-MARK QUESTIONS

1. Explain multimedia elements with examples.
2. Compare lossless and lossy compression.
3. Explain multimedia system architecture.
4. Explain need for data compression in multimedia.
5. List and explain multimedia applications.

10-MARK QUESTIONS

1. Explain multimedia elements in detail with properties, formats, and examples.
2. Explain multimedia system architecture with diagram.
3. Explain data compression techniques — lossless vs lossy, with examples of each.

🧠 Memory Tricks — Unit IV

- Multimedia elements: **T-I-A-V-A** = Text, Image, Audio, Video, Animation
- Lossless vs Lossy: **L**ossless = **L**etter-perfect; **L**ossy = **L**ess quality
- Compression redundancy types: **SPCT** = Spatial, Psychovisual, Coding, Temporal
- Lossy formats: **J-M-M** = JPEG, MPEG, MP3 (all popular lossy formats)
- MM Architecture layers from bottom: **N-I-H-O-S-U** = Network, I/O, Hardware, OS, Software, UI

V Media Streams & File Format Standards

Unit Overview:

- Covers multimedia streams (media, medium, properties), and all major file format standards: RTF, TIFF, RIFF, MIDI, JPEG, DIB, MPEG.
- Exam importance: 🔥 Very High — file formats are a staple question. Knowing full forms, purpose, and key characteristics is essential.

5.1 Media and Data Streams 🔥 Very Important

> **MEDIUM**
>
> A **medium** is the means by which information is stored or transmitted. In multimedia, it refers to the type of data: text, graphics, audio, video, or animation.

> **DATA STREAM**
>
> A **data stream** is a continuous sequence of data elements made available over time. In multimedia, streams carry audio, video, or other media data in real-time or near-real-time.

Types of Streams:

Type	Description	Example
Continuous Media Stream	Data that has temporal relationship between elements (must be delivered in time-order)	Audio, Video
Discrete Media Stream	Data elements with no timing relationship	Text, Images, Graphics
Live Stream	Real-time media captured and delivered immediately	Live broadcast, video call
On-Demand Stream	Pre-recorded media delivered on request	YouTube, Netflix

Main Properties of a Multimedia Stream

Property	Definition
Timeliness	Data must arrive and be processed within specific time bounds (real-time constraint)
Throughput (Bandwidth)	Rate of data delivery must match the media bit rate to avoid starvation
Reliability	Data must be delivered without errors (some loss may be acceptable for video)
Synchronization	Multiple streams (audio+video) must remain synchronized (lip sync)
QoS (Quality of Service)	Guaranteed levels of performance for delivery (latency, jitter, packet loss)
Jitter	Variation in packet arrival time — must be controlled for smooth playback
Latency	Delay between sending and receiving data — critical for live streams

Multimedia System Definition

> DEFINITION
>
> A **multimedia system** is defined as a computer system which handles multiple forms of digitally encoded media (text, graphics, audio, video, animation) in an integrated, synchronized, and interactive fashion.

Combination of Media:

- **Multimedia Presentation:** Multiple media combined for output (slide show with images + narration)
- **Hypermedia:** Non-linear multimedia with hyperlinks connecting different media (web pages)
- **Interactive Multimedia:** User can control the media flow (video games, interactive kiosks)
- **Synchronized Multimedia:** Audio and video tightly synchronized (movies, video calls)

5.2 Data & File Format Standards 🔥 Very Important

> FILE FORMAT
>
> A **file format** is a standard way of encoding and organizing data in a computer file. Each format has a specific structure (header + data + optional index) defined by a standard.

RTF, TIFF, RIFF 🔥 Very Important

RTF — Rich Text Format

> **FULL FORM:**
> Rich Text Format

- **Developed by:** Microsoft (1987)
- **Purpose:** Cross-platform document exchange with formatting (fonts, styles, colors, tables)
- **Type:** Text document format (not binary — human-readable control words)
- **Structure:** Plain ASCII text with **control words** preceded by backslash (e.g., `\b` for bold, `\i` for italic)
- **Extension:** .rtf
- **Application:** Word processing; cross-application document exchange
- **Advantage:** Readable by many word processors; preserves formatting across platforms
- **Limitation:** Larger than plain text; less feature-rich than DOCX or PDF

TIFF — Tagged Image File Format

> **FULL FORM:**
> Tagged Image File Format

- **Developed by:** Aldus Corporation (now Adobe), 1986
- **Purpose:** Storing high-quality raster images, especially for professional graphics and printing
- **Type:** Raster image format
- **Structure:** Header → Image File Directories (IFDs) → Tags with image data. Each **tag** holds a specific parameter (width, height, color depth, compression type)
- **Extension:** .tif, .tiff
- **Compression:** Supports multiple — uncompressed, LZW (lossless), JPEG (lossy)
- **Advantage:** High quality; flexible; supports multiple images in one file; lossless option
- **Limitation:** Large file size; not web-friendly
- **Application:** Desktop publishing, medical imaging, scanning, archival

RIFF — Resource Interchange File Format

> **FULL FORM:**
> Resource Interchange File Format

- **Developed by:** Microsoft and IBM (1991)
- **Purpose:** Container format for storing various types of multimedia data
- **Structure:** Chunk-based — each **chunk** has a 4-character identifier, size, and data. Chunks can be nested.
- **Child formats:** WAV (audio), AVI (audio+video), ANI (animated cursor), CDR
- **Extension:** varies (.wav, .avi, .rif)
- **Advantage:** Flexible extensible container; self-describing chunks
- **Limitation:** Limited to 4GB (32-bit size field); less efficient than modern containers

MIDI, JPEG/DIB, MPEG 🔥 Very Important

MIDI — Musical Instrument Digital Interface

> **FULL FORM:**
> Musical Instrument Digital Interface

- **Developed by:** MMA (MIDI Manufacturers Association), 1983
- **Purpose:** Communication protocol and file format for electronic musical instruments and computers
- **Key concept:** MIDI does NOT store audio; it stores **musical instructions** (note on/off, pitch, velocity, channel, duration)
- **Structure:** Header chunk + Track chunks; each track contains time-stamped MIDI events
- **File size:** Very small (few KB) — stores commands, not audio samples
- **Extension:** .mid, .midi
- **Advantage:** Tiny file size; editable at note level; supports multiple instruments
- **Limitation:** Sound quality depends on the synthesizer/soundfont used; not suitable for voice or natural sounds
- **Application:** Digital keyboards, music composition, game music, karaoke

JPEG — Joint Photographic Experts Group

> **FULL FORM:**
> Joint Photographic Experts Group (refers to both the committee and the format)

- **Standard:** ISO 10918; developed 1986–1992
- **Purpose:** Lossy compression of continuous-tone still images (photographs)
- **Type:** Lossy compression image format

- **Algorithm:** DCT (Discrete Cosine Transform) + Quantization + Huffman coding
- **Steps of JPEG compression:**
  1. Convert RGB to YCbCr (luminance + chrominance)
  2. Divide image into 8×8 pixel blocks
  3. Apply DCT to each block (spatial → frequency domain)
  4. Quantize (divide by quantization table — this is where loss occurs)
  5. Entropy encode (Huffman/arithmetic coding)
- **Extension:** .jpg, .jpeg
- **Quality factor:** 0–100 (higher = better quality, larger file)
- **Advantage:** High compression ratio (10:1 to 100:1); widely supported; good for photos
- **Limitation:** Lossy — repeated save degrades quality; poor for sharp edges/text; no transparency

DIB — Device Independent Bitmap

> **FULL FORM:**
> Device Independent Bitmap

- **Purpose:** A raster image format that stores color information independent of display device, so images appear consistently across different devices
- **Structure:** BITMAPFILEHEADER + BITMAPINFOHEADER + Color Table (optional) + Pixel Data
- **Extension:** .bmp, .dib
- **Advantage:** Simple, no compression needed; directly rendered by OS; universal support
- **Limitation:** Very large file sizes (uncompressed); not suitable for web
- **Application:** Windows native image format; clipboard; icons

MPEG Standards — Moving Picture Experts Group

> **FULL FORM:**
> Moving Picture Experts Group

A family of standards for audio and video compression, developed by ISO/IEC.

Standard	Purpose	Application	Resolution
MPEG-1	Video+audio compression for digital storage	Video-CD (VCD), MP3 audio	352×240
MPEG-2	Broadcast-quality video compression	DVD, Digital TV, HDTV broadcast	720×480 to 1920×1080
MPEG-4	Interactive and streaming multimedia	Internet video, mobile video, MP4, YouTube	Flexible
MPEG-7	Multimedia content description (metadata)	Content search and retrieval	—
MPEG-21	Multimedia framework for rights management	DRM, e-commerce of media	—

MPEG Video Frame Types:

- **I-frame (Intra):** Complete independent frame — largest; used as reference
- **P-frame (Predicted):** Encoded as difference from previous frame — medium size
- **B-frame (Bidirectional):** Encoded from both previous and next frames — smallest

> **Mnemonic:** I = **I**ndependent (full frame); P = **P**revious (uses past); B = **B**oth directions

📊 Master Comparison: All File Formats

Format	Full Form	Type	Compression	Extension	Best For
RTF	Rich Text Format	Text doc	None (text-based)	.rtf	Formatted text, word processing
TIFF	Tagged Image File Format	Raster image	None/LZW/JPEG	.tif	High-quality print, archival
RIFF	Resource Interchange File Format	Container	Varies (by child)	.wav/.avi	Audio (WAV), Video (AVI) container
MIDI	Musical Instrument Digital Interface	Music events	N/A (commands)	.mid	Electronic music, instruments
JPEG	Joint Photographic Experts Group	Raster image	Lossy (DCT)	.jpg	Photographs, web images
DIB	Device Independent Bitmap	Raster image	None (usually)	.bmp	Windows images, system icons
MPEG	Moving Picture Experts Group	Video/Audio	Lossy (DCT+motion)	.mpg/.mp4	Video streaming, DVD, broadcasting

📌 Quick Revision — Unit V

- Stream = continuous data sequence; audio/video = continuous media
- Stream properties: Timeliness, Bandwidth, Reliability, Sync, QoS, Jitter
- RTF = Rich Text Format; formatted text; Microsoft; .rtf
- TIFF = Tagged Image; professional images; LZW; .tif
- RIFF = container; WAV (audio), AVI (video) are RIFF children
- MIDI = musical commands (not audio); very small files; .mid
- JPEG = lossy; DCT-based; photographs; .jpg
- DIB = Device Independent Bitmap; Windows; uncompressed; .bmp
- MPEG-1=VCD; MPEG-2=DVD/TV; MPEG-4=Internet/mobile
- MPEG frame types: I (full), P (from past), B (bidirectional)

2-MARK QUESTIONS

1. What is MIDI?
2. Full form of MPEG and JPEG.
3. What is RTF?
4. What is RIFF?
5. What is DIB?
6. What are I, P, B frames in MPEG?

5-MARK QUESTIONS

1. Explain JPEG compression with steps.
2. Compare MIDI and MPEG standards.
3. Explain MPEG standards (1,2,4).
4. Explain properties of multimedia streams.
5. Compare RTF, TIFF and RIFF.

10-MARK QUESTIONS

1. Explain all file format standards: RTF, TIFF, RIFF, MIDI, JPEG, DIB, MPEG with comparison table.
2. Explain MPEG standards in detail with frame types.
3. Explain multimedia streams — medium, properties, combination of media.

🧠 Memory Tricks — Unit V

- File formats order: **ReTRIfied MIDI Jamming Drum Machine** = RTF, TIFF, RIFF, MIDI, JPEG, DIB, MPEG
- RIFF children: **W**AV and **A**VI = "**WA**tch AVI, listen WAV, all in RIFF"
- MIDI = **M**usical **I**nstructions, **D**igital **I**nterface (not audio!)
- JPEG steps: **Y-8-D-Q-H** = YCbCr → 8×8 blocks → DCT → Quantize → Huffman
- MPEG frames: **I P B** = **I**solated, **P**revious, **B**oth
- Stream properties: **T-B-R-S-Q-J-L** = Timeliness, Bandwidth, Reliability, Sync, QoS, Jitter, Latency

Formula Sheet

SCAN CONVERSION — LINES

Formula	Variables	Algorithm
steps = max(|Δx|, |Δy|)	$\Delta x=x_2-x_1$, $\Delta y=y_2-y_1$	DDA
x_inc = Δx/steps; y_inc = Δy/steps	Increments per step	DDA
d_initial = 2·dy − dx	$dy=y_2-y_1$, $dx=x_2-x_1$	Bresenham Line
If d<0: d = d + 2·dy (E pixel)	Choose East pixel	Bresenham Line
If d≥0: y++; d = d + 2(dy−dx) (SE pixel)	Choose SE pixel	Bresenham Line

SCAN CONVERSION — CIRCLES

Formula	Variables
p_initial = 1 − r	r = radius
If p<0: x++; p = p + 2x + 1	East decision
If p≥0: x++; y--; p = p + 2x − 2y + 1	SE decision
8-fold symmetry: plot (±x, ±y) and (±y, ±x)	8 points from 1

SCAN CONVERSION — ELLIPSES

Formula	Use
$f(x,y) = b^2x^2 + a^2y^2 - a^2b^2$	Ellipse function (0=on ellipse)
$p_1_initial = b^2 - a^2b + a^2/4$	Region 1 initial dp
Region 1 → Region 2 when: $2b^2x \geq 2a^2y$	Transition condition
4-fold symmetry: plot (±x, ±y)	4 points from 1

CLIPPING — COHEN-SUTHERLAND

Region codes: LEFT=0001, RIGHT=0010, BOTTOM=0100, TOP=1000

Boundary	Condition	Intersection Formula
Left (x=xmin)	x < xmin	$y = y_1 + m\cdot(xmin - x_1)$
Right (x=xmax)	x > xmax	$y = y_1 + m\cdot(xmax - x_1)$
Bottom (y=ymin)	y < ymin	$x = x_1 + (ymin - y_1)/m$
Top (y=ymax)	y > ymax	$x = x_1 + (ymax - y_1)/m$

Slope: $m = (y_2-y_1)/(x_2-x_1)$

CYRUS-BECK

$P(t) = P_1 + t\cdot(P_2 - P_1)$, $t \in [0, 1]$

$t = [N_i \cdot (PE_i - P_1)] / [N_i \cdot (P_2 - P_1)]$

$D = N_i \cdot (P_2-P_1)$: D<0 → entering (tL=max); D>0 → leaving (tU=min)

If tL > tU → reject. Else draw P(tL) to P(tU).

2D TRANSFORMATIONS

Transform	Formula	3×3 Matrix
Translation	x'=x+tx; y'=y+ty	|1,0,tx; 0,1,ty; 0,0,1|
Rotation (CCW)	x'=xcosθ−ysinθ; y'=xsinθ+ycosθ	|cosθ,−sinθ,0; sinθ,cosθ,0; 0,0,1|
Scaling	x'=x·Sx; y'=y·Sy	|Sx,0,0; 0,Sy,0; 0,0,1|
Reflection (X-axis)	x'=x; y'=−y	|1,0,0; 0,−1,0; 0,0,1|
Reflection (Y-axis)	x'=−x; y'=y	|−1,0,0; 0,1,0; 0,0,1|
Shear-X	x'=x+Shx·y; y'=y	|1,Shx,0; 0,1,0; 0,0,1|

WINDOW-TO-VIEWPORT TRANSFORMATION

sx = (vxmax−vxmin) / (wxmax−wxmin)

sy = (vymax−vymin) / (wymax−wymin)

xv = vxmin + (xw − wxmin) · sx

yv = vymin + (yw − wymin) · sy

3D TRANSFORMATIONS (4×4 MATRICES)

Translation T(tx,ty,tz): last column = $[tx, ty, tz, 1]^T$

Scaling S(Sx,Sy,Sz): diagonal = [Sx, Sy, Sz, 1]

Rotation-Z: same as 2D rotation extended with 1s for z and w

Rotation-X: cosθ and ±sinθ in the y-z submatrix

Rotation-Y: cosθ and ±sinθ in the x-z submatrix (note sign reversal)

🎯 MASTER EXAM REVISION

> **How to use this section:** This is your *last-stop revision* before the exam. Read all headings, scan tables, and focus on 🔥 items.

A. Most Important Topics

Unit	🔥 Must Know
I	Bresenham's Line (with numerical) · Midpoint Circle · Midpoint Ellipse · Conceptual Framework · Scan Conversion
II	Cohen-Sutherland (region codes + numerical) · Cyrus-Beck (parametric) · Raster vs Random-Scan
III	All 5 transformations with matrices · Homogeneous coordinates · Window-to-Viewport (numerical) · Composition
IV	Multimedia elements (T-I-A-V-A) · Lossless vs Lossy · Multimedia System Architecture
V	JPEG (DCT steps) · MPEG (standards + I/P/B frames) · MIDI · All file formats comparison table

B. Most Expected 10-Mark Questions

1. 🔥 Explain Bresenham's line drawing algorithm with derivation and numerical example.
2. 🔥 Explain Cohen-Sutherland line clipping algorithm with region codes, diagram, and numerical.
3. 🔥 Explain all 2D transformations with homogeneous matrices and numerical examples.
4. 🔥 Explain multimedia elements in detail with properties, formats, and comparison.
5. 🔥 Explain file format standards: RTF, TIFF, RIFF, MIDI, JPEG, DIB, MPEG with comparison table.
6. ⭐ Explain midpoint circle algorithm with steps and numerical.
7. ⭐ Explain midpoint ellipse algorithm (both regions).
8. ⭐ Explain Window-to-Viewport transformation with derivation and numerical.
9. ⭐ Explain multimedia system architecture with diagram.
10. ⭐ Explain data compression — lossless vs lossy — with algorithms and multimedia importance.

11. ⭐ Explain Cyrus-Beck parametric clipping algorithm.
12. ○ Explain 3D transformation matrices in detail.

C. Most Expected 5-Mark Questions

1. Compare DDA and Bresenham's line algorithm.
2. Explain 8-fold symmetry in circle scan conversion.
3. Compare raster-scan and random-scan displays.
4. Explain Cohen-Sutherland region codes.
5. Compare Cohen-Sutherland and Cyrus-Beck algorithms.
6. Explain composition of transformations with example.
7. Explain homogeneous coordinates and why they are used.
8. Explain multimedia applications.
9. Compare lossless and lossy compression.
10. Explain MPEG standards (MPEG-1, 2, 4).
11. Explain JPEG compression process (DCT steps).
12. Explain multimedia streams and their properties.

D. Most Expected 2-Mark Questions (with answers)

Question	Short Answer
What is computer graphics?	Creating, manipulating, storing, displaying visual info using computers.
What is scan conversion?	Converting geometric primitives (lines, circles) to discrete pixel representations on raster displays.
Initial decision parameter in Bresenham's line?	d = 2·dy – dx
Initial decision parameter in Bresenham's circle?	p = 1 – r
What is 8-fold symmetry?	A circle is symmetric in 8 octants, so computing one point gives 7 others automatically.
What is clipping?	Removing portions of primitives outside the clip window.
Inside region code (Cohen-Sutherland)?	0000
Trivial rejection condition?	Bitwise AND of both outcodes ≠ 0
Why homogeneous coordinates?	To represent all 2D transforms (including translation) as matrix multiplication.
What is a viewport?	The rectangular area on the screen where the clipped window content is displayed.
Window-to-Viewport scale factor formula?	sx = (vxmax–vxmin)/(wxmax–wxmin)
What is multimedia?	Integration of text, images, audio, video, and animation in a computer-based system.
What is lossless compression?	Compression where the original data is perfectly reconstructed on decompression.
Full form of MIDI?	Musical Instrument Digital Interface
Full form of MPEG?	Moving Picture Experts Group

Full form of JPEG?	Joint Photographic Experts Group
Full form of TIFF?	Tagged Image File Format
Full form of RIFF?	Resource Interchange File Format
Full form of DIB?	Device Independent Bitmap
What are I, P, B frames in MPEG?	I=Intra (full frame); P=Predicted (from past); B=Bidirectional (from past+future)
What is DCT?	Discrete Cosine Transform — converts spatial image data to frequency domain for compression.
What is a frame buffer?	Memory storing pixel intensity values for the complete current display image.

E. Important Algorithms Summary

Algorithm	Purpose	Key Parameter	Type
DDA Line	Draw line	x_inc = Δx/steps	Floating-point
Bresenham Line	Draw line	d_init = 2dy−dx	Integer
Midpoint Circle	Draw circle	p_init = 1−r; 8 symmetry	Integer
Midpoint Ellipse	Draw ellipse	2 regions; 4 symmetry	Integer
Cohen-Sutherland	Clip line (rectangle)	4-bit outcodes; AND/OR tests	Region code
Cyrus-Beck	Clip line (convex polygon)	$P(t)=P_1+t(P_2-P_1)$; tL, tU	Parametric
Midpoint Subdivision	Clip line	Bisect at midpoint recursively	Recursive

F. Important Diagrams to Draw

1. CG Conceptual Framework pipeline (2 rows: output + input)
2. DDA and Bresenham step table for a given line
3. Circle plotted using Bresenham with 8-fold symmetry
4. Cohen-Sutherland 9-region diagram with outcodes

5. Raster-scan vs Random-scan working principle
6. Window-to-Viewport mapping (two rectangles with labels)
7. 2D transformation examples (point before/after)
8. Multimedia System Architecture (layered diagram)
9. JPEG compression steps (flowchart)
10. MPEG I/P/B frame sequence

17 One-Day Revision Sheet

Revise the entire subject in one session. One line per concept.

UNIT I — Introduction & Scan Conversion

- CG = creating/displaying visual info with computers
- Interactive CG: user ↔ system real-time
- Scan conversion = geometric shape → discrete pixels
- DDA: floating-point; steps=max(|Δx|,|Δy|); x_inc=Δx/steps
- Bresenham Line: integer; d_init=2dy−dx; d<0→E; d≥0→SE+y++
- Bresenham Circle: p_init=1−r; 8-fold symmetry; integer ops
- Ellipse: 4-fold symmetry; 2 regions; region switch: $2b^2x \geq 2a^2y$
- Conceptual Framework: App→API→System→Display (+ Input loop)

UNIT II — Devices & Clipping

- Raster-scan: row-by-row pixels; frame buffer; 60Hz refresh
- Random-scan: draws only vectors; sharp lines; display list
- Frame buffer = all pixel values stored in memory
- Video controller: reads frame buffer → drives monitor
- Cohen-Sutherland: 4-bit outcodes; trivial accept (0000 OR=0); reject (AND≠0)
- Region codes: L=0001,R=0010,B=0100,T=1000; inside=0000
- Intersection: left boundary→$y=y_1+m(x_{min}-x_1)$; top→$x=x_1+(y_{max}-y_1)/m$
- Cyrus-Beck: $P(t)=P_1+t(P_2-P_1)$; t=W/D; tL=max, tU=min
- Midpoint Subdivision: bisect; trivial test; repeat until ε small

UNIT III — Transformations

- Translation: x'=x+tx, y'=y+ty
- Rotation CCW: x'=xcosθ−ysinθ; y'=xsinθ+ycosθ
- Scaling: x'=x·Sx; y'=y·Sy
- Reflection X-axis: (x,−y); Y-axis: (−x,y); origin: (−x,−y)
- Shear-X: x'=x+Shx·y; y'=y
- Homogeneous: 2D point → (x,y,1); all transforms = 3×3 matrix multiply

Composition: multiply matrices in order; order matters!

Rotate about P(px,py): T(−px,−py) → R(θ) → T(px,py)

W→V: sx=(vx_range)/(wx_range); xv=vxmin+(xw−wxmin)·sx

3D: 4×4 matrices; Rz=2D extended; Rx,Ry in y-z, x-z submatrices

UNIT IV — Multimedia

Multimedia = T-I-A-V-A (Text, Image, Audio, Video, Animation)

Audio: sampling rate × bit depth × channels × time / 8 = size

Video: fps × resolution × color depth = data rate

MM Architecture: UI→Software→OS→Hardware→I/O→Network

Compression needed: 1 min HD = 3.7 GB uncompressed

Lossless: exact recovery (PNG, TIFF-LZW, GIF)

Lossy: approximate (JPEG, MP3, MPEG); small files

RLE: AAABBB→3A3B; Huffman: short code for frequent symbol

MMDB: stores multimedia + metadata; content-based retrieval

UNIT V — Streams & File Formats

Stream: continuous media (audio/video) vs discrete (text/image)

Stream properties: Timeliness, Bandwidth, Reliability, Sync, QoS, Jitter

RTF: Rich Text Format; MS 1987; formatted text; .rtf; control words

TIFF: Tagged Image; professional; LZW/JPEG; .tif; tag-based

RIFF: container; chunk-based; WAV=audio child; AVI=video child

MIDI: musical commands (NOT audio); tiny files; .mid; note on/off events

JPEG: lossy; DCT; 8×8 blocks; quality 0-100; .jpg; photos

DIB: Device Independent Bitmap; Windows; uncompressed; .bmp

MPEG-1=VCD; MPEG-2=DVD/TV; MPEG-4=Internet/mobile video

MPEG frames: I=full; P=from previous; B=bidirectional (smallest)

✅ Last-Minute Revision Checklist

Go through each item before your exam:

- ☐ Bresenham Line — write d_init, two cases, trace one example
- ☐ Midpoint Circle — write p_init, two cases, 8 symmetry points
- ☐ Midpoint Ellipse — Region 1 and Region 2 rules
- ☐ DDA vs Bresenham comparison table (4 rows)
- ☐ Cohen-Sutherland: 9-region diagram; 4-bit codes; 3 cases
- ☐ Cohen-Sutherland intersection formulas (4 boundaries)
- ☐ Cyrus-Beck: P(t) formula; D<0→tL; D>0→tU
- ☐ All 5 transformation matrices (3×3 homogeneous)
- ☐ Window-to-Viewport formula and one numerical
- ☐ 3D rotation matrices for X, Y, Z axes
- ☐ Multimedia 5 elements — T-I-A-V-A with formats
- ☐ Lossless vs Lossy table + examples of each
- ☐ Multimedia System Architecture — 6-layer diagram
- ☐ All file format full forms + 2-line description
- ☐ MPEG-1, 2, 4 differences + I/P/B frames
- ☐ JPEG compression steps (5 steps: YCbCr→8×8→DCT→Q→Huffman)

ALL THE BEST! 🌟

Computer Graphics & Multimedia · BCA Semester V · BCAMACO501T22

Jaipur National University, Jaipur · Complete Topper Notes